

EX. NO: 1 a

**PYTHON PROGRAM USING SIMPLE STATEMENTS AND EXPRESSIONS -
EXCHANGE THE VALUES OF TWO VARIABLES**

PROGRAM:

```
a=int(input("Enter the value of A: "))
b=int(input("Enter the value of B: "))
a,b=b,a
print("The swapping of a value is ",a)
print("The swapping of b value is ",b)
```

OUTPUT:

```
Enter the value of A: 5
Enter the value of B: 4
The swapping of a value is 4
The swapping of b value is 5
```

EX. NO: 1 b

**PYTHON PROGRAM USING SIMPLE STATEMENTS AND EXPRESSIONS -
CALCULATE THE DISTANCE BETWEEN TWO POINTS**

PROGRAM:

```
import math
x1=int(input("Enter the value of x1 = "))
x2=int(input("Enter the value of x2 = "))
y1=int(input("Enter the value of y1 = "))
y2=int(input("Enter the value of y2 = "))
dx=x2-x1
dy=y2-y1
d=dx**2+dy**2
result=math.sqrt(d)
print(result)
```

OUTPUT:

```
Enter the value of x1= 5
Enter the value of x2=6
Enter the value of y1=6
Enter the value of y2=3
3.1622776601683795
```

EX NO: 2 a

**SCIENTIFIC PROBLEMS USING CONDITIONALS AND ITERATIVE LOOPS
- NUMBERSERIES**

PROGRAM:

```
n=int(input(" Enter the number of terms for the series "))  
  
i=1  
x=0  
while(i<=n):  
    x=x*2+1  
    print(x) i+=1
```

OUTPUT:

```
Enter the number of terms  
for the series 5 1  
3  
7  
15  
31
```

EX. NO: 2 b

**SCIENTIFIC PROBLEMS USING CONDITIONALS AND ITERATIVE LOOPS –
NUMBER PATTERNS**

PROGRAM:

```
rows = int(input('Enter the number of rows'))
for i in range(rows+1):
    for j in range(i):
        print(i, end = ' ')
    print(' ')
```

OUTPUT:

```
Enter the number of rows5 1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

EX NO: 2 c

**SCIENTIFIC PROBLEMS USING CONDITIONALS AND ITERATIVE LOOPS
- PYRAMID**

PROGRAM:

```
n = int(input('Enter the number of rows'))
for i in range(0,n):
    for j in range(0,i+1):
        print("*", end = " ")
    print('\n')
```

OUTPUT:

```
Enter the no. of rows 5
*
* *
* * *
* * * *
* * * * *
```

EX NO: 3 a

**IMPLEMENTING REAL-TIME/TECHNICAL APPLICATIONS USING LISTS, TUPLES
– PRINT ITEMS PRESENT IN A LIBRARY**

PROGRAM:

```
library=["books", "author", "barcodenumber" , "price"]
library[0]="Let us C"
print(library[0])
library[1]=" Yashavant Kanetkar"
library[2]=123987
library[3]=234 print(library)
tup1 = (12134,250000 )
tup2 = ('books', 'totalprice')
tup3 = tup1 + tup2;
print(tup3)
```

OUTPUT:

```
Let us C
['Let us C', 'Yashavant Kanetkar', 123987, 234] (12134, 250000, 'books',
'totalprice')
```

EX NO: 3 b

**IMPLEMENTING REAL-TIME/TECHNICAL APPLICATIONS USING LISTS, TUPLES
– PRINT SYSTEM NAMES**

PROGRAM:

```
print("\nLISTS:")
Computers = ["Dell", "Lenovo", "HP", "Apple", "Acer", "Asus"]
new_list = []
for i in Computers: if "A" in i:
    new_list.append(i)
    print(new_list)
print("\nTUPLE:")
Computers=("Dell", "Lenovo", "HP", "Apple", )
print(Computers)
print(Computers [0])
print(Computers [1])
print(Computers [2])
print(Computers [3])
```

OUTPUT:

LISTS:

```
['Apple'] ['Apple', 'Acer']
['Apple', 'Acer', 'Asus']
```

TUPLE:

```
('Dell', 'Lenovo', 'HP', 'Apple')
Dell
Lenovo
HP
Apple
```

EX NO: 3 c

**IMPLEMENTING REAL-TIME/TECHNICAL APPLICATIONS USING LISTS, TUPLES
- MATERIALS REQUIRED FOR CONSTRUCTION OF A BUILDING**

PROGRAM:

```
print("\nLISTS:")
materials= ["cementbags", "bricks", "sand", "Steelbars", "Paint"]
materials.append(" Tiles")
materials.insert(3,"Aggregates")
materials.remove("sand")
materials[5]="electrical"
print(materials)
print("\nTUPLES:")
materials = ("cementbags", "bricks", "sand", "Steelbars", "Paint")
print(materials)
print ("list of element is=",materials)
print ("materials[0]:", materials [0])
print ("materials[1:3]:", materials [1:3])
```

OUTPUT:

LISTS:

```
['cementbags', 'bricks', 'Aggregates', 'Steelbars', 'Paint', 'electrical']
```

TUPLES:

```
('cementbags', 'bricks', 'sand', 'Steelbars', 'Paint')
list of element is= ('cementbags', 'bricks', 'sand', 'Steelbars', 'Paint')
materials[0]: cementbags
materials[1:3]: ('bricks', 'sand')
```

EX NO: 4**IMPLEMENTING REAL-TIME/TECHNICAL APPLICATIONS USING SETS AND
DICTIONARIES****PROGRAM:**

```
print("\nSETS:")
cars = {'BMW', 'Honda', 'Audi', 'Mercedes', 'Honda', 'Toyota', 'Ferrari', 'Tesla'}
print('Approach #1= ', cars)
print('=====')
print('Approach #2')
for car in cars:
    print('Car name = {}'.format(car))
    print('=====')
    cars.add('Tata')
print('New cars set = {}'.format(cars))
cars.discard('Mercedes')
print('discard() method = {}'.format(cars))
print("\nDICTIONARY:")
Dict = {}
print("Empty Dictionary: ")
print(Dict)
# Adding elements one at a time
Dict[0] = 'BRICKS'
Dict[2] = 'CEMENT'
Dict[3] = 'BLUE PRINT'
print("\nDictionary after adding 3 elements: ")
print(Dict)
# Adding set of values # to a single Key
Dict['Value_set'] = 2, 3, 4
print("\nDictionary after adding 3 elements: ")
print(Dict)
# Updating existing Key's Value
Dict[2] = 'STEEL'
print("\nUpdated key value: ")
print(Dict)
# Adding Nested Key value to Dictionary
Dict[5] = {'Nested': {'1': 'LIME', '2': 'SAND'}}
print("\nAdding a Nested Key: ")
print(Dict)
```

OUTPUT:**SETS:**

```
Approach#1= {'BMW', 'Mercedes', 'Toyota', 'Audi', 'Ferrari', 'Tesla', 'Honda'}
=====
Approach #2
Car name = BMW
Car name = Mercedes
Car name = Toyota
Car name = Audi
```

Car name = Ferrari

Car name = Tesla

Car name = Honda

=====

New cars set = {'BMW', 'Mercedes', 'Toyota', 'Audi', 'Ferrari', 'Tesla', 'Honda', 'Tata'}

discard() method = {'BMW', 'Toyota', 'Audi', 'Ferrari', 'Tesla', 'Honda', 'Tata'}

DICTIONARY:

Empty Dictionary: {}

Dictionary after adding 3 elements:

{0: 'BRICKS', 2: 'CEMENT', 3: 'BLUE_PRINT'}

Dictionary after adding 3 elements:

{0: 'BRICKS', 2: 'CEMENT', 3: 'BLUE_PRINT', 'Value_set': (2, 3, 4)}

Updated key value:

{0: 'BRICKS', 2: 'STEEL', 3: 'BLUE_PRINT', 'Value_set': (2, 3, 4)}

Adding a Nested Key:

{0: 'BRICKS', 2: 'STEEL', 3: 'BLUE_PRINT', 'Value_set': (2, 3, 4), 5: {'Nested': {'1': 'LIME', '2': 'SAND'}}}

EX NO: 5 a

IMPLEMENTING FACTORIAL PROGRAMS USING FUNCTIONS

PROGRAM:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)  
n=int(input("Input a number to compute the factorial : "))  
print(factorial(n))
```

OUTPUT:

Input a number to compute the factorial: 4

24

EX NO: 5 b

**IMPLEMENTING PROGRAMS USING FUNCTIONS – FIND THE LARGEST NUMBER
IN A LIST**

PROGRAM:

```
def max_num_in_list( list ): max = list[ 0 ]
    for a in list:
        if a > max:
            max = a
    return max
print(max_num_in_list([1, 2, -8, 0]))
```

OUTPUT:

2

EX NO: 5 c**IMPLEMENTING PROGRAMS USING FUNCTIONS – AREA OF SHAPES
PROGRAM:**

```
# define a function for calculating
# the area of a shapes
def calculate_area(name):
    name = name.lower()
    # check for the conditions
    if name == "rectangle":
        l = int(input("Enter rectangle's length: "))
        b = int(input("Enter rectangle's breadth: "))
        # calculate area of rectangle
        rect_area = l * b
        print(f"The area of rectangle is {rect_area}.")
    elif name == "square":
        s = int(input("Enter square's side length: "))
        # calculate area of square
        sqt_area = s * s
        print(f"The area of square is {sqt_area}.")
    elif name == "triangle":
        h = int(input("Enter triangle's height length: "))
        b = int(input("Enter triangle's breadth length: "))
        # calculate area of triangle
        tri_area = 0.5 * b * h
        print(f"The area of triangle is {tri_area}.")
    elif name == "circle":
        r = int(input("Enter circle's radius length: "))
        pi = 3.14
        # calculate area of circle
        circ_area = pi * r * r
        print(f"The area of circle is {circ_area}.")
    elif name == "parallelogram":
        b = int(input("Enter parallelogram's base length: "))
        h = int(input("Enter parallelogram's height length: "))
        # calculate area of parallelogram
        para_area = b * h
        print(f"The area of parallelogram is {para_area}.")
    else:
        print("Sorry! This shape is not available")
print("Calculate Shape Area")
name = input("Enter the name of shape whose area you want to find: ")
calculate_area(name)
```

OUTPUT:

Calculate Shape Area

Enter the name of shape whose area you want to find: rectangle

Enter rectangle's length: 10

Enter rectangle's breadth: 15

The area of rectangle is 150.

EX NO: 6 a

IMPLEMENTING PROGRAMS USING STRINGS – REVERSE

PROGRAM:

```
def reverse(string):  
    string = string[::-1]  
    return string  
  
s =input("Enter a string: ")  
print ("The original string is ",end="")  
print (s)  
print ("The reversed string(using extended slice syntax) is ",end="")  
print (reverse(s))
```

OUTPUT:

```
Enter a string: Firstyearcse  
The original string is Firstyearcse  
The reversed string(using extended slice syntax) is escaeYtsrIF
```

EX NO: 6 b

IMPLEMENTING PROGRAMS USING STRINGS - PALINDROME

PROGRAM:

```
def isPalindrome(s):  
    return s == s[::-1]  
    # Driver code  
s = input("Enter the string: ") ans = isPalindrome(s)  
if ans:  
    print("The string is palindrome ")  
else:  
    print("The string is not a palindrome")
```

OUTPUT:

Enter a string: madam

The string is a palindrome

EX NO: 6 c

**IMPLEMENTING PROGRAMS USING STRINGS - CHARACTER
COUNT**

PROGRAM:

```
input_string = input("Enter a string: ")  
  
count = 0  
  
for c in input_string :  
    if c.isspace() != True:  
        count = count + 1  
print("Total number of characters: ",count)
```

OUTPUT:

```
Enter a string: Hello, world!  
Total number of characters: 13
```